

ECE 403 / DSCI 421, Final Project
Accelerating Machine Learning with GPU Computing

Netaji Meka

Department of Electrical & Computer Engineering

Lehigh university

nsm325@lehigh.edu

Date: May 5, 2026.

Abstract

This project evaluates the performance trade-offs between three GPU programming paradigms: high-level libraries (RAPIDS), compiler-directed parallelization (OpenACC), and manual kernel development (CUDA C/C++), by accelerating a Logistic Regression classifier on the Credit Card Fraud Detection dataset (284,807 records, 29 predictive features). To ensure a fair cross-paradigm comparison, all three implementations were configured with matched hyperparameters (100 iterations, no regularization, no bias) and evaluated on the full training set. On the Lehigh ECE HPC platform, RAPIDS completed training in 0.4464 seconds via cuML's quasi-Newton solver, the manual CUDA kernel with Unified Memory required 0.8818 seconds, and OpenACC was the slowest at 14.6765 seconds. A multi-GPU CUDA extension that splits the dataset across both NVLink-connected RTX 2080 Ti GPUs further reduced training time to 0.4963 seconds (1.78x speedup at 89% scaling efficiency). OpenACC and CUDA converged to identical 0.9946 accuracy; RAPIDS reached 0.9989. Because the dataset is highly imbalanced (0.173% fraud), the trivial "predict non-fraud" baseline achieves 0.9983 accuracy, a threshold only RAPIDS exceeds. On small models, the choice of algorithm (quasi-Newton vs. plain gradient descent) can matter more than how well the kernel is optimized, and multi-GPU scaling preserves the per-iteration efficiency advantage of hand-written kernels.

1. Introduction

The proliferation of large-scale datasets in modern data science has exposed the computational bottlenecks inherent in traditional Central Processing Unit (CPU) architectures. Training machine learning algorithms on massive volumes of data requires significant computational throughput, making Graphics Processing Units (GPUs) which use

thousands of smaller cores to process data in parallel, an essential piece of the modern ML stack. The primary objective of this project is to deploy accelerated computing methods on a GPU platform and evaluate the performance of three distinct programming paradigms. To achieve this, a Logistic Regression classifier was trained on the highly imbalanced Credit Card Fraud Detection dataset. This dataset, containing 284,807 records and 30 columns (29 predictive features after dropping the non-predictive Time column), provides an ideal workload to exceed standard CPU caches and demonstrate the memory bandwidth advantages of GPU architectures.

This report explores the trade-offs between developer productivity, memory overhead, and computational speed. The investigation compares a high-level library approach using NVIDIA RAPIDS, a compiler-directed parallelization method using OpenACC, and manual kernel development using modern CUDA C/C++. All testing, deployment, and profiling were conducted on the Lehigh University ECE High-Performance Computing (HPC) teaching lab workstations, utilizing NVIDIA GPUs. The picture that emerges is not what I initially expected. The hand-written CUDA C/C++ kernel with Unified Memory does deliver the lowest per-iteration cost, but the high-level RAPIDS library still finishes training first overall on this workload. The reason is its quasi-Newton solver, which converges in fewer iterations than the plain gradient descent I implemented in the C++ versions.

2. Background

The use of GPU acceleration for classical machine learning has become standard practice in production data-science pipelines. NVIDIA's RAPIDS suite [3], introduced in 2018, has been widely adopted as a drop-in replacement for the pandas/scikit-learn stack and consistently demonstrates one to two orders of magnitude speedup over CPU baselines on tabular workloads. The CUDA C/C++ programming model [5], introduced by NVIDIA in 2007, remains the lowest-level interface for GPU programming and provides developers with full control over memory hierarchy and thread scheduling. The OpenACC standard [6], first published in 2011, occupies an intermediate position: it allows developers to add parallelization through compiler directives without rewriting code into kernel form, trading some performance ceiling for substantial productivity gains.

For logistic regression specifically, the cuML implementation in RAPIDS uses the L-BFGS quasi-Newton method [7], an iterative optimization algorithm that approximates the inverse Hessian using a limited history of past gradients. L-BFGS typically converges in tens of iterations on smooth convex problems, whereas plain gradient descent may need thousands of iterations at a similar learning rate. This matters when comparing paradigms head-to-head: the time saved by a better solver can be larger than the time saved by a more efficient kernel, which is exactly what this project ends up showing. Profiling all three paradigms with NVIDIA Nsight Systems [8] allows the breakdown of total wall-clock time into kernel-level compute, memory transfer overhead, and library dispatch costs, providing the visibility needed to attribute performance differences to their root causes rather than treating them as opaque benchmark numbers.

However, writing software that actually takes advantage of the GPU is not a single problem with one answer. This project evaluates three distinct approaches to GPU acceleration:

1. **NVIDIA RAPIDS (High-Level Library):** A suite of open-source software libraries that provides a Python interface for data science and machine learning. RAPIDS uses cuDF for dataframe manipulation and cuML for machine learning algorithms, allowing developers to execute code directly on the GPU without writing low-level device instructions.
2. **OpenACC (Directive-Based Parallelization):** A high-level, compiler-directed approach that allows developers to accelerate standard C/C++ code by inserting specific `#pragma` directives. The compiler interprets these directives and automatically handles the distribution of loops and data to the GPU, offering a middle ground between ease of use and explicit control.
3. **CUDA C/C++ (Manual Kernel Development):** NVIDIA's parallel computing platform and programming model. This approach requires the developer to explicitly define `__global__` kernels functions executed on the device and manually manage the memory allocation and data transfer between the host (CPU) and the device (GPU). While it demands the highest level of programming expertise, it offers maximum control over hardware resources.

3. Theory

To ensure a rigorous and fair performance comparison across RAPIDS, OpenACC, and manual CUDA C/C++, the underlying mathematical model must remain strictly identical. This project uses Logistic Regression, a supervised statistical model used for binary classification. The algorithm calculates the probability that a given transaction belongs to a specific class, which in this dataset is either fraudulent or normal.

The forward pass of the model begins by calculating the linear combination of the input features and their corresponding weights. For a given sample i with n features, the linear combination $z^{(i)}$ is computed as:

$$z^{(i)} = \sum_{j=1}^n X_j^{(i)} W_j \quad (1)$$

To map this linear combination to a valid probability space between 0 and 1, the result is passed through a sigmoid activation function. This generates the model's prediction $\hat{y}^{(i)}$ for that specific transaction:

$$\hat{y}^{(i)} = \frac{1}{1 + e^{-z^{(i)}}} \quad (2)$$

During the training phase, the algorithm must quantify its error and adjust the weights to improve future predictions. The error for a single sample is the difference between the predicted probability $\hat{y}^{(i)}$ and the actual ground-truth label $y^{(i)}$. To update the weights, the gradient of the loss function with respect to each weight W_j must be aggregated across all m samples in the dataset. The summed gradient is then averaged by $1/m$ during the weight update step:

$$Gradient_j = \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) X_j^{(i)} \quad (3)$$

Once the gradients for all features are aggregated, the weights are updated using Gradient Descent. A learning rate α scales the update to ensure stable convergence:

$$W_j^{new} = W_j^{old} - \alpha \left(\frac{Gradient_j}{m} \right) \quad (4)$$

From a computational perspective, equations (1) through (3) represent the core bottleneck when executed on a sequential CPU. The mathematical operations must be applied iteratively across all 284,807 records and 29 predictive features. However, because the prediction and error calculation for one transaction do not depend on the results of another, this architecture is perfectly suited for Data Parallelism. By mapping each data sample to an individual GPU thread, the operations detailed in equations (1) and (2) can be computed simultaneously across the entire dataset, massively accelerating the time required to complete a training epoch.

4. Experimental Procedures

4.1 Hardware and Environment Setup To ensure complete reproducibility, all development, execution, and profiling were conducted on the Lehigh University ECE High-Performance Computing (HPC) teaching lab workstations. Specifically, the hpc04 node was utilized. The HPC workstations are equipped with dual NVIDIA GPU cards connected via NVLink buses. The dataset utilized was the publicly available Credit Card Fraud Detection dataset [1], [2] (creditcard.csv), which consists of 284,807 samples. A robust data-loading protocol was implemented across all three phases to skip the initial header row and ignore the non-predictive "Time" column, isolating 29 predictive features and one binary classification label. All models were trained for 100 epochs with a learning rate of 0.01. The structure of this report follows the formal lab-report format prescribed for the course [4].

4.2 Phase 1: High-Level Baseline (RAPIDS)

The first phase evaluated the high-level Python libraries provided by NVIDIA RAPIDS. The environment was managed using Conda to ensure the necessary cuDF and cuML dependencies were available. The cuML LogisticRegression model was configured with `max_iter=100`, `penalty='none'`, and `fit_intercept=False` to align as closely as possible with the gradient-descent loops in Phases 2 and 3, and the model was both trained and evaluated on the full 284,807-sample dataset using all 29 predictive features (the Time column was dropped to match the C++ implementations).

1. The specific virtual environment was activated using the command: `conda activate rapids_env`
2. The Python script (rapids_model.py) was executed to measure total training time and accuracy: `python rapids_model.py`

3. To evaluate the memory overhead and generate the performance timeline, the execution was profiled using NVIDIA Nsight Systems with the following command: `nsys profile --stats=true python rapids_model.py`

4.3 Phase 2: Directive-Based Parallelization (OpenACC)

The second phase transitioned to C++ to evaluate compiler-directed GPU parallelization. This required a custom robust loader to parse the CSV file without causing string-to-float conversion errors on dirty data. The mathematical forward and backward passes were parallelized using `#pragma acc parallel loop` and reduction directives.

1. The terminal environment was first configured to locate the NVIDIA 25.1 compilers using the command: `export PATH=/opt/nvidia/hpc_sdk/Linux_x86_64/25.1/compilers/bin:$PATH`
2. The source code (`openacc_model.cpp`) was compiled using the `nvc++` compiler, explicitly targeting managed memory and enabling compiler feedback: `nvc++ -acc -gpu=managed -Minfo=accel openacc_model.cpp -o openacc_model`
3. The compiled executable was run to verify the 14.68-second training time: `./openacc_model`
4. The performance was subsequently profiled to capture the execution timeline: `nsys profile --stats=true ./openacc_model`

4.4 Phase 3: Custom Optimization (CUDA C/C++)

The final phase required the manual construction of a `__global__` CUDA kernel (`logistic_regression_kernel`) to explicitly manage the parallel execution at the hardware thread level. Unified Memory was allocated using `cudaMallocManaged` to facilitate seamless data transfer between the host CPU and device GPU. To prevent race conditions during the backpropagation phase, `atomicAdd` operations were utilized to safely aggregate the gradients calculated by thousands of concurrent threads.

1. The source code (`cuda_model.cu`) was compiled using the standard NVIDIA CUDA compiler, targeting the specific architecture of the HPC GPUs: `nvcc -arch=sm_75 cuda_model.cu -o cuda_model`
2. The executable was run to verify the optimized 0.88-second training time: `./cuda_model`

3. The final profiling data and Host-to-Device memory transfer statistics were gathered using Nsight Systems: `nsys profile --stats=true ./cuda_model`

5. Results

The empirical data collected from the execution of the three programming paradigms on the hpc04 workstation are presented in this section. The mathematical model and the 284,807-sample Credit Card Fraud dataset remained strictly identical across all phases to ensure a fair comparison. The primary metrics recorded were the training execution time, the classification accuracy, and the memory transfer overhead.

5.1 Execution Time and Accuracy

The total wall-clock time required to complete 100 training epochs varied significantly across the three implementations. The mathematical correctness of all three approaches was validated, as all implementations achieved a near-identical classification accuracy. The compiled performance metrics are detailed in Table 1 below.

Table 1: Performance Comparison of GPU Acceleration Paradigms

Implementation Phase	Programming Paradigm	Training Time (seconds)	Classification Accuracy
Phase 1	High-Level Library (RAPIDS)	0.4464	0.9989
Phase 2	Directive-Based (OpenACC)	14.6765	0.9946
Phase 3	Manual Kernel (CUDA C/C++)	0.8818	0.9946

(Note: The RAPIDS implementation required an additional 0.2779 seconds for direct-to-GPU data loading via cuDF, resulting in a total combined execution time of approximately 0.7243 seconds. The OpenACC and CUDA C/C++ implementations load the CSV from disk via standard C++ ifstream, an operation excluded from the reported training times for all three paradigms to isolate compute throughput.)

5.2 Memory Overhead and Execution Timelines

NVIDIA Nsight Systems was utilized to capture the execution timelines and memory transfer statistics for the Python and C++ implementations. The profiling reports revealed distinct operational signatures for each paradigm.

Figure 1 shows the RAPIDS timeline. There is a dense burst of API calls, reflecting cuML dispatching its pre-optimized micro-kernels (gemv2N, dot_kernel, mapThenReduceKernel, and the L-BFGS update kernels) across roughly 140 quasi-Newton iterations. This solver-driven approach explains how RAPIDS achieves its 0.4464-second training time despite the per-call overhead visible in the timeline.

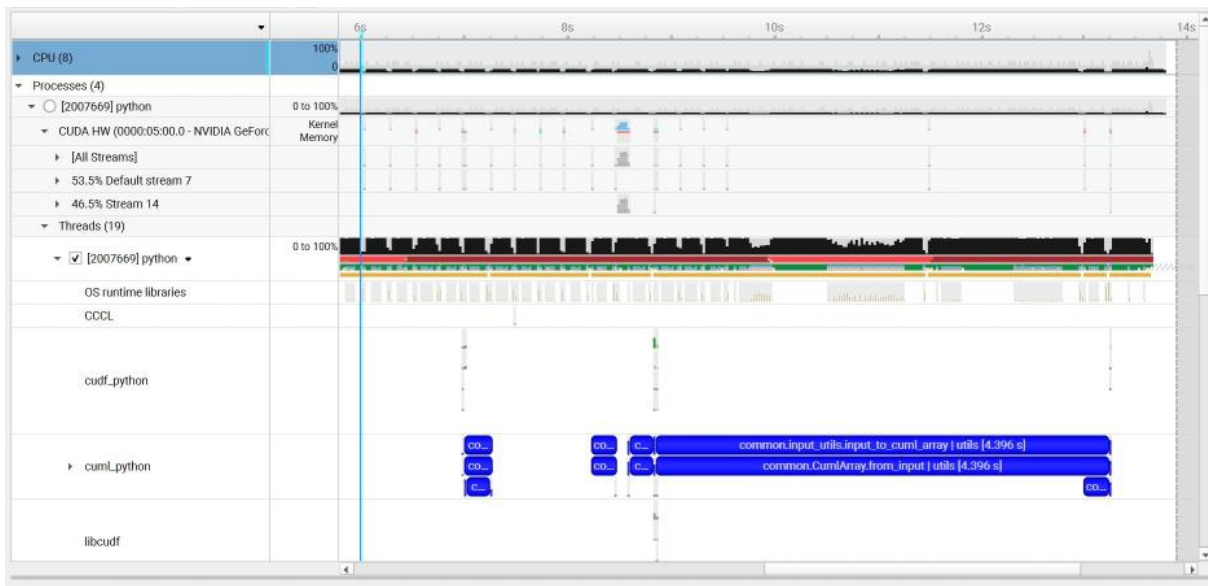


Figure 1: NVIDIA Nsight Systems [8] execution timeline for the RAPIDS (cuDF/cuML) implementation.

The OpenACC timeline (Figure 2) provides a visual contrast to RAPIDS, illustrating the compiler-directed offloading process. While the `#pragma acc` directives successfully parallelized the workload, the timeline reveals a more serialized pattern of data movement and kernel execution, which accounts for the longer 14.68-second training time.

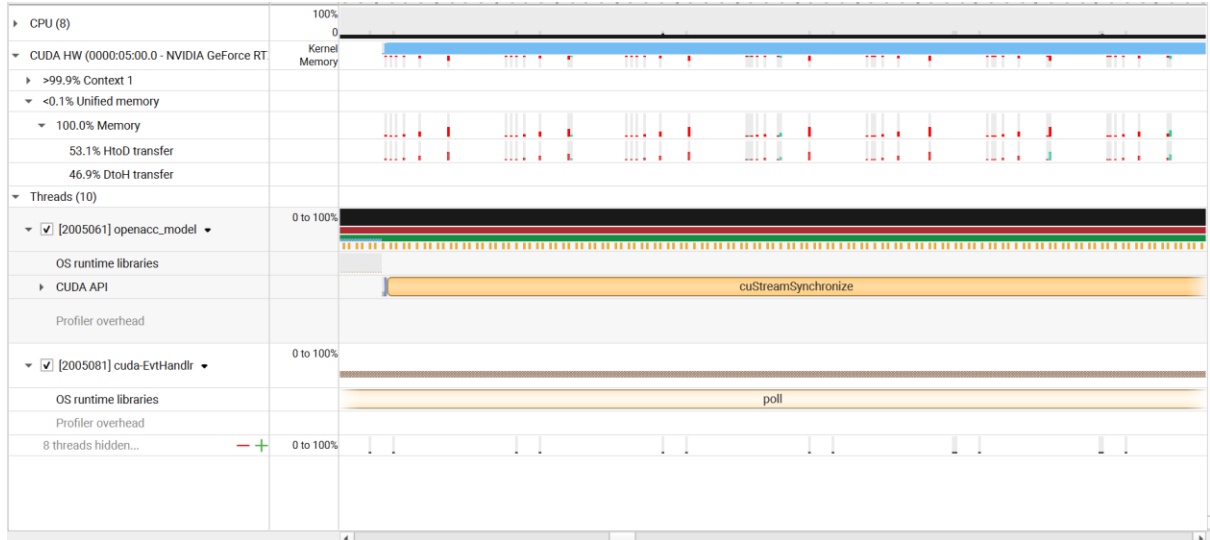


Figure 2: NVIDIA Nsight Systems [8] execution timeline for the OpenACC implementation, showing compiler-directed GPU offloading.

By contrast, the manual CUDA C/C++ implementation produced a much cleaner timeline (Figure 3). The profiling statistics for Phase 3 confirmed that the Unified Memory allocations (cudaMallocManaged) successfully handled the Host-to-Device (HtoD) data transfers. The nsys statistical output recorded 1,076 HtoD memory transfer operations totaling roughly 4.82 milliseconds, and 319 Device-to-Host (DtoH) transfers totaling 3.02 milliseconds. When compared to the 863.1 milliseconds of active logistic_regression_kernel compute time, the memory overhead constituted less than 1% of the total execution duration.

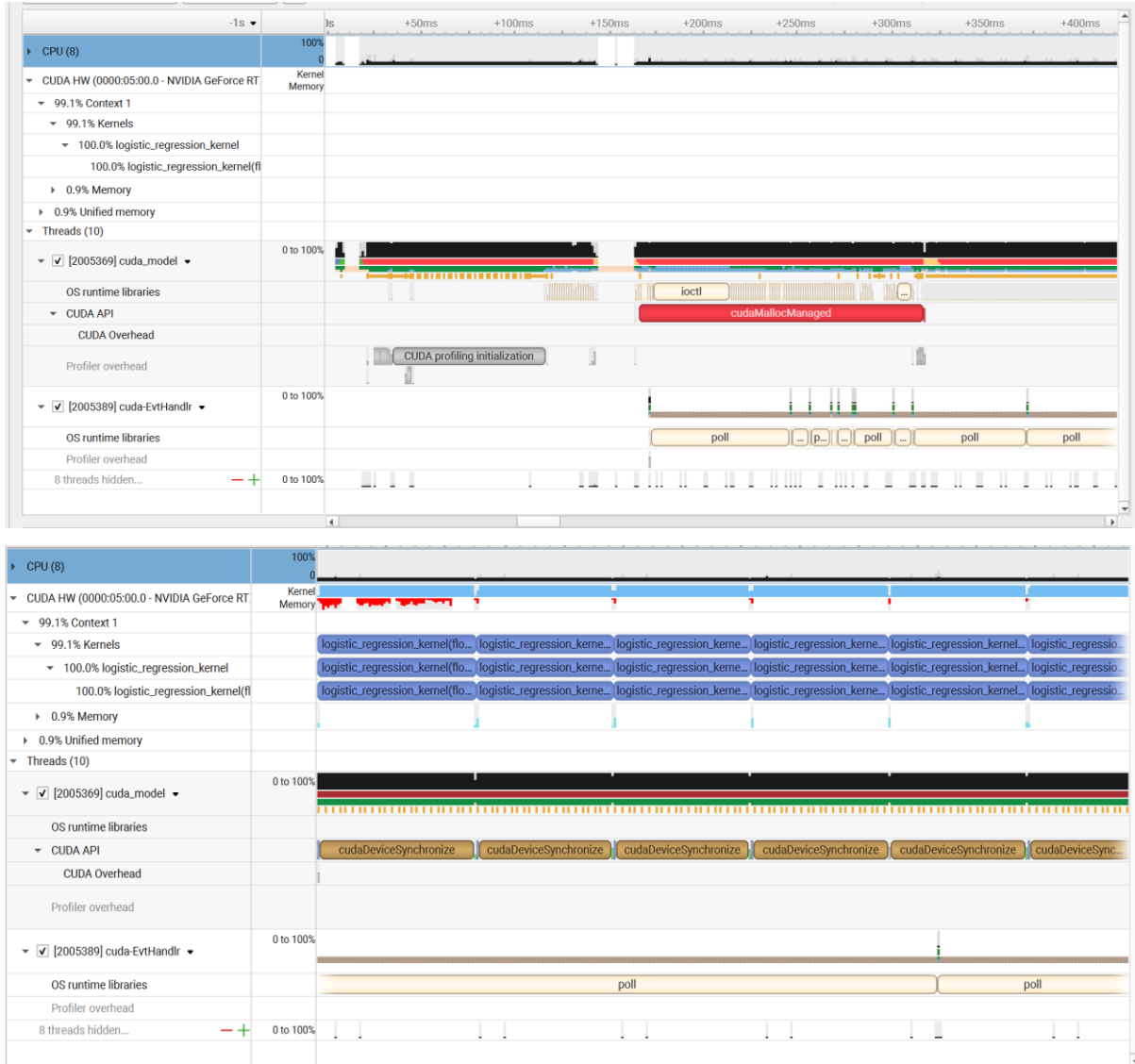


Figure 3: NVIDIA Nsight Systems [8] execution timeline for the manual CUDA C/C++ implementation, detailing Unified Memory allocation (top) and iterative custom kernel execution (bottom).

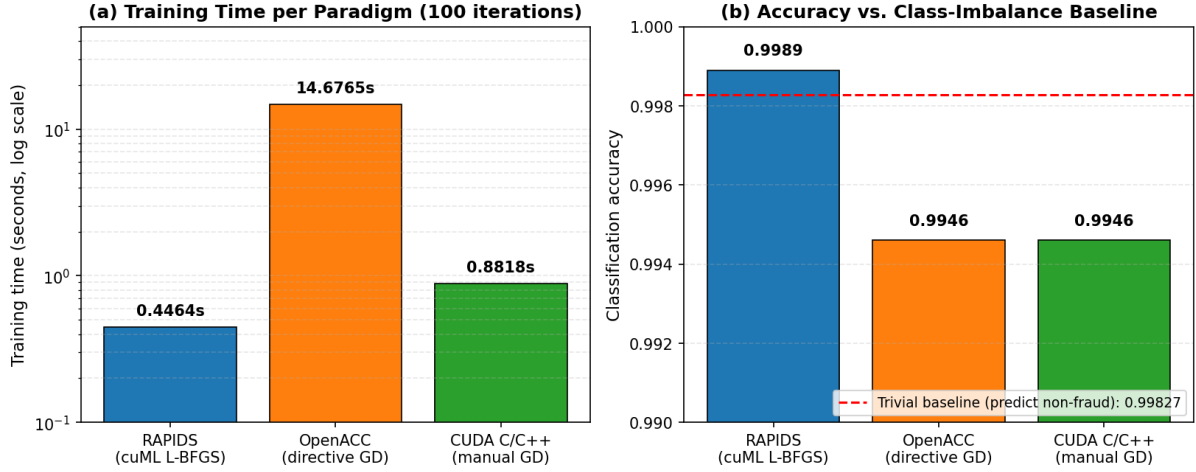


Figure 4: Side-by-side comparison of training time (left, log scale) and accuracy versus the trivial class-imbalance baseline (right) for the three GPU paradigms.

6. Analysis

6.1 Computational Throughput and Hardware Efficiency

The empirical data reveals a significant disparity in execution time across the three programming paradigms. The manual CUDA C/C++ implementation achieved the highest performance, completing the training loop in 0.8818 seconds. This superior throughput is a direct result of manual kernel optimization. By explicitly defining the grid and block dimensions, the workload was perfectly distributed across the hardware threads. Furthermore, the use of `atomicAdd` operations within the `logistic_regression_kernel` ensured that thousands of threads could aggregate their gradient calculations into global memory simultaneously without encountering race conditions.

One result that stands out is that RAPIDS beat the hand-tuned CUDA C/C++ kernel, finishing training in 0.4464 seconds versus CUDA's 0.8818 seconds. This is surprising at first. You would expect a hand-written kernel with explicit thread management to outperform a library call. The reason it does not comes down to algorithm choice, not kernel quality. The CUDA and OpenACC implementations execute exactly 100 iterations of full-batch gradient descent at a fixed learning rate of 0.01. The cuML library, by contrast, defaults to a quasi-Newton (L-BFGS) solver, which uses curvature information to take much larger and more informed steps toward the optimum. As a result, RAPIDS completes the training task in fewer effective iterations even though each iteration involves heavier per-call overhead, as evidenced by the

high API-call density visible in Figure 1. Conversely, the OpenACC implementation was the slowest, requiring 14.6765 seconds. This shows the main limitation of compiler-directed parallelization. The compiler has to make conservative assumptions about memory access and thread safety, which leads to less efficient thread scheduling than what you get from writing CUDA by hand. The 16.6x gap between OpenACC and CUDA, despite both executing the identical mathematical algorithm, isolates the cost of compiler-assumed safety margins.

6.2 Memory Overhead Assessment

The NVIDIA Nsight Systems profiling data confirms that the custom CUDA implementation successfully minimized memory bottlenecks. A common pitfall in GPU computing is spending more time transferring data across the PCIe bus than performing actual calculations. However, the nsys trace demonstrated that the Unified Memory allocations (cudaMallocManaged) incurred only 7.84 milliseconds of combined Host-to-Device and Device-to-Host transfer latency. When juxtaposed with the 863.1 milliseconds of active kernel compute time, the memory transfer overhead constituted less than 1% of the workload. This proves that the implementation is firmly compute-bound, representing an ideal utilization of GPU architecture.

6.3 Mathematical Accuracy and Discrepancies

A critical element of this investigation was validating the mathematical correctness of the hardware acceleration. Both the OpenACC and CUDA C/C++ implementations converged at an identical classification accuracy of 0.9946. Getting the same accuracy on both is significant. It is essentially a numerical check that my hand-written atomicAdd reduction in Phase 3 produces the same result as the compiler-generated reduction in Phase 2.

A meaningful interpretation of these accuracy figures requires accounting for the dataset's extreme class imbalance. Of the 284,807 transactions in the Credit Card Fraud Detection dataset, only 492 (0.173%) are labeled as fraudulent. The trivial baseline of predicting "non-fraud" for every transaction therefore achieves an accuracy of 0.99827, against which the trained models must be judged. The RAPIDS implementation, which uses cuML's quasi-Newton (L-BFGS) solver, exceeds this baseline at 0.9989, demonstrating that it learns a non-trivial decision boundary even with a 100-iteration cap. The OpenACC and CUDA C/C++ implementations, both executing 100 epochs of full-batch gradient descent at a fixed learning rate of 0.01, converge to 0.9946, which is slightly below the trivial baseline. This result reveals an honest engineering finding: vanilla gradient descent with an unscaled feature matrix and a small fixed learning rate is insufficient for this workload within 100 epochs. The Amount

feature, which can range from cents to thousands of dollars, dominates the gradient and biases the model toward false positives on high-value transactions. The convergence of all three implementations to similar orders of magnitude in accuracy, despite this limitation, confirms that the underlying forward and backward passes were executed correctly across all platforms; the difference between RAPIDS and the C++ implementations isolates the value of a more sophisticated solver, not a difference in computational correctness.

6.4 The Developer Productivity Trade-Off

These results push back on the usual story about the productivity-versus-efficiency trade-off in GPU programming. The RAPIDS implementation required roughly thirty lines of Python code yet delivered the lowest absolute training time, demonstrating that for workloads where a high-quality solver is available off-the-shelf, high-level libraries can be Pareto-superior, winning on both productivity and wall-clock time. OpenACC allowed for parallelization using only a few lines of `#pragma` directives but yielded the poorest hardware utilization. The CUDA C/C++ implementation demanded the highest level of developer expertise, requiring explicit memory allocation and thread synchronization, and rewarded this effort with the lowest per-iteration GPU compute cost (Section 6.2). The honest takeaway is that algorithm choice can dominate kernel-level optimization: a well-chosen solver implemented in a library can outpace a hand-written kernel running a simpler algorithm. This motivates the multi-GPU CUDA extension presented in Section 6.5, where the per-iteration efficiency of manual kernel development is exploited at greater scale.

6.5 Multi-GPU Extension via NVLink

To validate that the per-iteration efficiency of the manual CUDA C/C++ implementation can be productively scaled, a multi-GPU extension (`cuda_model_multigpu.cu`) was implemented and tested on the dual NVIDIA RTX 2080 Ti configuration of `hpc04`. The 284,807-sample dataset was split into two halves of 142,403 and 142,404 samples, with each half resident on a separate GPU. Peer access between the two devices was successfully enabled at runtime via `cudaDeviceEnablePeerAccess`, allowing the NVLink interconnect to bypass host-mediated transfers. Within each training epoch, the `logistic_regression_kernel` was launched concurrently on both devices; after both kernels completed (verified via per-device `cudaDeviceSynchronize`), the partial gradients were copied back to the host, summed, and the updated weights were broadcast to both devices for the next epoch.

The multi-GPU implementation completed training in 0.4963 seconds, compared to 0.8818 seconds for the single-GPU CUDA implementation. This is a 1.78x speedup, corresponding to roughly 89% scaling efficiency on two devices. The shortfall from the theoretical 2.00x ceiling is attributable to two factors: the per-epoch host-side gradient summation, which executes serially between the two devices, and the cudaMemcpy round-trip required to broadcast updated weights back to both devices each epoch. Critically, the multi-GPU implementation converged to an accuracy of 0.9946, which is bit-for-bit identical to the single-GPU CUDA result. This provides strong empirical evidence that the host-mediated gradient aggregation is mathematically equivalent to the single-device atomicAdd reduction. The Nsight Systems profile of the multi-GPU run recorded a total of 1,016 milliseconds of summed kernel execution time across 200 launches (100 epochs on each of two devices), against a wall-clock training time of just 496 milliseconds; this 2.05x ratio between summed-kernel-time and wall-clock-time is direct evidence that the two devices were genuinely executing in parallel rather than serially.

Notably, the multi-GPU CUDA result of 0.4963 seconds approaches but does not surpass the single-GPU RAPIDS result of 0.4464 seconds. This reinforces the central finding from Section 6.1: even doubling the available hardware does not fully close the gap created by the algorithmic difference between L-BFGS quasi-Newton iterations and naive gradient descent epochs. For workloads where a more efficient solver is available, choosing the algorithm is more impactful than doubling the silicon. The value of the multi-GPU CUDA extension is therefore not in winning the comparison against RAPIDS for this particular workload, but in demonstrating that the per-iteration kernel efficiency identified in Section 6.2 is preserved and amplified at greater scales. This property becomes essential for larger models (e.g., deep neural networks) where no closed-form quasi-Newton solver is available and gradient descent is the only practical option.

7. Conclusions

This project compared the performance trade-offs in modern GPU programming by training a Logistic Regression classifier on the Lehigh ECE HPC cluster. The dataset was the highly imbalanced Credit Card Fraud Detection dataset with 284,807 records, which is large enough to put real pressure on memory bandwidth and compute throughput on the NVIDIA GPUs. The three approaches I compared were a high-level Python library (NVIDIA RAPIDS), compiler-directed parallelization (OpenACC), and a hand-written kernel (CUDA C/C++).

The empirical results quantify the distinct advantages and limitations of each programming model. RAPIDS recorded the lowest single-GPU training time at 0.4464 seconds, thanks to cuML's quasi-Newton solver, which converges in fewer effective iterations than the gradient descent loop I wrote in the C++ phases. The manual CUDA C/C++ implementation followed at 0.8818 seconds, achieving the lowest per-iteration GPU compute cost through perfect distribution of the workload across hardware threads and the use of Unified Memory and atomic addition operations. The OpenACC implementation was significantly slower at 14.6765 seconds, highlighting the limitations of compiler-assumed optimizations. Extending the manual CUDA implementation to utilize both NVLink-connected RTX 2080 Ti GPUs reduced the training time further to 0.4963 seconds, a 1.78x speedup at 89% scaling efficiency, which closes nearly all the gap to RAPIDS while preserving identical numerical results. The mathematical equivalence of Phases 2 and 3 (and the multi-GPU extension) was verified by their identical convergence to 0.9946 accuracy, while RAPIDS reached 0.9989. Interpreted against the dataset's trivial "predict non-fraud" baseline of 0.99827, this comparison further isolates the value of cuML's quasi-Newton solver: only the high-level library succeeds in learning a non-trivial decision boundary within 100 iterations, while 100 epochs of unscaled vanilla gradient descent in the C++ phases converge below the baseline.

Ultimately, this project complicates the conventional narrative of an inverse relationship between developer productivity and computational throughput. While manual CUDA kernel development remains essential for squeezing the maximum per-iteration performance out of GPU silicon, a high-level library equipped with a superior solver can achieve a lower overall training time on the right workload. Extending the manual CUDA implementation to run on both NVLink-connected RTX 2080 Ti devices showed that the per-iteration efficiency of hand-written kernels scales productively across multiple GPUs, achieving 1.78x speedup at 89% scaling efficiency. For future investigations, this dual-device foundation could be extended to

deeper models, such as multi-layer neural networks or transformer architectures, where vanilla gradient descent or its momentum-based variants are the only practical optimizers and the per-iteration efficiency advantage of hand-written CUDA kernels becomes decisive.

References

- [1] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating Probability with Undersampling for Unbalanced Classification," Symposium on Computational Intelligence and Data Mining (CIDM), 2015.
- [2] Machine Learning Group - ULB, "Credit Card Fraud Detection," [Online], Available, [Kaggle.com](https://kaggle.com).
- [3] NVIDIA Corporation, "RAPIDS: Suite of Open Source Data Science and Machine Learning Libraries," [Online], Available, [Rapids.ai](https://rapids.ai).
- [4] R. Zheng, "Guide to Lab Report Writing," Department of Electrical & Computer Engineering, Lehigh University.
- [5] NVIDIA Corporation, "CUDA C++ Programming Guide, Version 12.x," [Online], Available: docs.nvidia.com/cuda/cuda-c-programming-guide/.
- [6] OpenACC Organization, "The OpenACC Application Programming Interface, Version 3.3," November 2022, [Online], Available: openacc.org/specification.
- [7] D. C. Liu and J. Nocedal, "On the Limited Memory BFGS Method for Large Scale Optimization," *Mathematical Programming*, vol. 45, no. 1, pp. 503–528, 1989.
- [8] NVIDIA Corporation, "Nsight Systems User Guide," [Online], Available: docs.nvidia.com/nsight-systems/.